IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 8 July 2022, accepted 2 September 2022, date of publication 8 September 2022, date of current version 19 September 2022.

Digital Object Identifier 10.1109/ACCESS.2022.3204976

RESEARCH ARTICLE

Edge Workloads Monitoring and Failover: A StarlingX-Based Testbed Implementation and Measurement Study

MOHAMMED ABUIBAID[1], AMIR HOSSEIN GHORAB[1], AIDAN SEGUIN-MCPEAKE[2], OWEN YUEN[2], THOMAS YUNGBLUT[2], AND MARC ST-HILAIRE[1,2], (Senior Member, IEEE)

[1]Department of Systems and Computer Engineering, Carleton University, Ottawa, ON K1S 5B6, Canada
[2]School of Information Technology, Carleton University, Ottawa, ON K1S 5B6, Canada

Corresponding author: Mohammed Abuibaid (mohammedaa.abuibaid@carleton.ca)

ABSTRACT With the ever-growing amount of time-critical, compute-intensive, and private IoT applications, the need for High Availability (HA) Edge Clouds becomes indispensable. Realizing HA Edge Clouds is inherently challenging due to the geographically-dispersed hierarchy of the Distributed Cloud Infrastructure (DCI). For example, frequent isolation between the central Cloud and Edge Clouds due to networking instability necessitates some autonomous operations at the Edge Clouds. Furthermore, because Edge Clouds have fewer resources than central Clouds, configuring the Edge functions (i.e., control, compute, and storage) in HA clusters will undoubtedly reduce downtime. However, it will limit the Edge scalability. To that end, StarlingX is developing an HA-protected and scalable DCI virtualization platform based on the open-source ecosystem, focusing on low-touch management of Edge Clouds. StarlingX provides a fault management service that realizes DCI-wide alarming and logging capabilities, allowing for rapid response to virtualized infrastructure events. Recently, the IETF Network Working Group proposed that monitoring both the DCI and the Edge workloads (software containers) is critical for an Edge Computing Platform to maintain HA IoT application deployment. Indeed, the possibility of the infrastructure remaining stable and healthy while the workloads suffer a fatal failure simultaneously necessitates failover functionality that monitors both the infrastructure and the Edge workloads. In this paper, we first propose a dynamic failover functionality that centrally monitors Edge workloads to recover from deployment or Edge node failures, motivated by the IETF direction. Second, we experimentally optimize the failover functionality for monitoring a microservice-architected IoT application deployed on a StarlingX-based DCI testbed to collect temperature sensor readings from Raspberry Pis. Regardless of how quickly the Edge workload health checks are collected, the recorded failover measurements reveal that the recovery time will not drop below a predetermined level controlled by Edge resources and network speed. Furthermore, reducing the statistics collection timeout reduces the recovery time of an Edge node failure. When the timeout value is less than the minimum achievable recovery time, false-positive failures (FPFs) can occur. Third, to supplement the StarlingX fault management service, we provide a modular implementation of the proposed failover functionality. Finally, we present the first-ever introduction of the StarlingX platform's software stack to promote its use in academic research.

INDEX TERMS Distributed cloud infrastructure, edge computing, failover, IoT, Kubernetes, microservice architecture, StarlingX platform, testbed.

The associate editor coordinating the review of this manuscript and approving it for publication was Sathish Kumar.

I. INTRODUCTION

Edge computing calls to perform downstream and upstream data computation at the network Edge on behalf of the central Cloud data centers and Edge devices, respectively [1]. Such a paradigm brings mutual benefits to all stakeholders. Particularly, end-users get a lower latency for time-critical services by leveraging the nearby computing resources. Data privacy becomes less challenging since user data can be restricted from travelling to central Clouds. The user equipment computing and energy constraints are relaxed by offloading computation tasks to Edge servers. Lastly, from a network operator perspective, Edge computing releases the pressure on transport links connecting users to the central Cloud.

A. CHALLENGES OF EDGE CLOUDS MANAGEMENT

Due to the geographically dispersed nature of the Edge Clouds in a Distributed Cloud Infrastructure (DCI), managing Edge workloads and devices presents three additional challenges.

1) CHALLENGE 1 - EDGE CLOUDS SCALABILITY

Because the number of Edge devices can be enormous, for example, in massive IoT deployments for smart agriculture or industrial automation, so does their management overhead. Controlling this overhead is crucial since Edge Cloud devices and sites are always thought to have limited resources (e.g., hardware-restricted CPU, RAM, storage, and networking capabilities) or physical space restrictions (e.g., power supply and personal access).

2) CHALLENGE 2 - EDGE CLOUD AUTONOMY

The Edge Clouds of a DCI are frequently isolated from the central Cloud, or occasionally, the communication throughput and latency change ambiguously. In contrast to the centralized Cloud computing paradigm, where data centre computing resources are physically aggregated within a few to hundred metres. Therefore, it is essential to maintain operations at the Edge Clouds and synchronization reliability until the connectivity is restored.

3) CHALLENGE 3 - EDGE CLOUDS INTER-COLLABORATION

The Edge Clouds are intended to collaborate vertically with the central Cloud rather than horizontally with neighbouring Edge Clouds. This design creates isolated computing environments at the edges and eliminates any potential benefits of inter-edge collaborations. Enabling such collaboration is advantageous for providing alternative resources at central Clouds that are subject to unreliable communication. For example, inter-Edge collaboration can offload congested Edge Clouds. Alternatively, it may maintain low-latency services when central Clouds are located far more than neighbour Edge Clouds.

Taking on these Edge Cloud management challenges imposes new requirements that differ depending on the specific Edge use case. For example, restoring inaccessible Edge workloads is critical for any DCI failover mechanism. For example, when an Edge computing node fails, the central Cloud controller should seamlessly relocate the affected workloads to another available Edge node to ensure service continuity. Furthermore, monitoring time-sensitive IoT workloads should have a small footprint on Edge Clouds. At the same time, failure recovery mechanisms should act quickly with zero-touch supervision.

B. StarlingX: A BRIEF OVERVIEW

StarlingX is a relatively new and constantly evolving open-source project — led by the Open Infrastructure Foundation[1] — to implement the Edge reference architecture with a distributed control plane [2]. It provides a central Cloud controller for managing and orchestrating thousands of geographically distributed Edge data centers [3]. Moreover, it offers high availability (HA)-protected Distributed Cloud infrastructure (DCI). The central or Edge Cloud[2] can run an HA cluster for controller, worker or storage functions. For instance, the StarlingX controller can be part of a two-node HA control cluster for running control functions in active/active or active/standby modes. The infrastructure failover is well-established and can undertake maintenance procedures, like updating hardware or replacing faulty components.

StarlingX provides workload container orchestration by leveraging Kubernetes clusters with both master and worker nodes at the Edge Clouds. Workload reliability and stability are maintained in each Edge Cloud by Kubernetes self-healing, auto-scaling, and dynamic scheduling features. However, the StarlingX community has not yet addressed the collaboration of Kubernetes clusters across Edge Clouds. Deploying container workloads across multiple Edge Clouds necessitates using a federation of Kubernetes clusters, which is being developed as part of the open-source Kubefed[3] project. Furthermore, the StarlingX platform does not currently provide monitoring or failover services for workloads distributed across Edge Clouds. Edge workload failures and the underlying Edge infrastructure are unknown to the central Cloud.

C. CONTRIBUTIONS

The primary goals of this paper are to introduce StarlingX to the Edge computing research community, implement a three-tier DCI testbed, and propose an Edge workload failover mechanism. To be more specific, the contributions are as follows:

[1] https://openinfra.dev/

[2] Unless specified, the term Edge Cloud, aka Sub-Cloud, refers to bare metal or virtual machine (VM)-based StarlingX platform deployment.

[3] KubeFed stands for Kubernetes Cluster Federation. Note: This project is currently in beta version (i.e., contains most of the major features but is not yet complete), https://github.com/kubernetes-sigs/kubefed

1) INTRODUCTION TO StarlingX

To the best of our knowledge, this is the first academic paper that introduces the StarlingX platform and describes its architectural design in detail. Our article is aimed at all researchers interested in learning more about the platform's fundamental building blocks derived from the open-source ecosystem. More importantly, it shows how StarlingX bridges the gaps between them to create a complete DCI virtualization platform for bare metal, VM, and container-based workloads. Although the StarlingX platform is prominent in online content, no single academic publication (i.e., conference or journal papers) is centred on it. Throughout our literature review, we discovered that StarlingX is either mentioned briefly in the introductions or appears in future works. This contribution is presented in Sec. III.

2) REAL DCI TESTBED IMPLEMENTATION

Using three geographically distributed and VPN-connected virtual servers, we provide a VM-based testbed implementation of a StarlingX-based distributed control DCI. With virtual environments, DCI experimentation is more affordable and accessible. Our testbed reduces the burden of providing dedicated physical servers as stipulated in the official StarlingX installation guidelines for a bare metal DCI [4]. In addition to this testbed, we are developing an IoT application that supports microservice architecture, distributed cloud deployments, and data collection from IoT devices (e.g., Raspberry Pi). Finally, we promote research reproducibility by making the testbed implementation, IoT application, and failover experimentation findings publicly available on GitHub.[4] This contribution is presented in Sec. IV.

3) POLLING-BASED FAILOVER MECHANISM

By proposing a polling-based failover functionality as detailed in Sec. V, we can monitor both the underlying Edge infrastructure and the Edge workloads to maintain HA IoT deployments. We contribute to the IETF general model of Edge Computing platform [5]. We demonstrate experimentally in Sec. VI how to reduce RTO for edge workloads or edge node failures. We examine the RTO for two scenarios: a failure of a microservice-architected IoT application and an unreachable Edge node. Then, we optimize the failover procedure using the data collected to achieve the shortest possible RTO.

D. PAPER ORGANIZATION

We organize the rest of this paper as follows. In Sec. II, we discuss the related work of DCI implementations and Edge workload failover mechanisms. Sec. III introduces the StarlingX platform. In Sec. IV, we describe both StarlingX-based DCI testbed and developed IoT application. Then, we introduce the proposed failover functionality in Sec. V. In Sec. VI, we discuss the experimental setup and collected measurements. Finally, we conclude the study in Sec. VII.

[4]https://github.com/MohammedAbuibaid/Stx_DCI_Failover

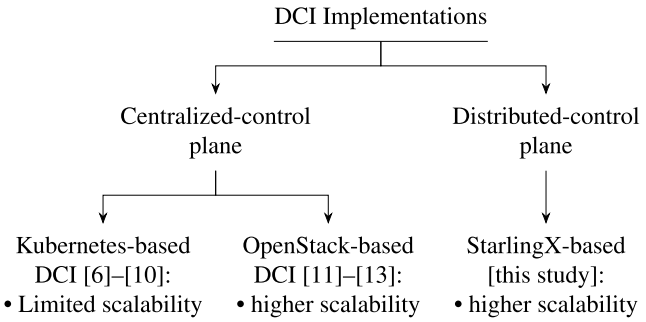


FIGURE 1. The DCI control approaches and implementations taxonomy.

II. RELATED WORK

Sarmiento *et al.* present a recent survey of SDN-based decentralized DCI management technologies. The authors emphasized the need for a DCI platform that automatically configures, provisions, and manages Edge devices in a scalable, resilient, and simple manner.

Several DCI implementations involving IoT deployments emphasised the DCI benefits to Edge computing use cases [6], [7], [8], [9], [10], [11], [12], [13]. The implemented DCIs in these studies, as shown in Fig. 1, use either a centralized-control plane or a distributed-control plane, as well as OpenStack Cloud Platform and Kubernetes Container Orchestration System. The following sections discuss the advantages and disadvantages of both approaches. The limitations of existing DCI implementations are then presented in preparation for the introduction of StarlingX, which follows a distributed-control plane approach and leverages both OpenStack and Kubernetes.

A. CENTRALIZED CONTROL PLANE DCI IMPLEMENTATIONS

1) KUBERNETES-BASED DCI IMPLEMENTATIONS

The authors of [6], [7], and [8] realized that by containerizing and distributing Edge workloads among worker nodes, they could handle complex machine learning and video analytics applications [6] as well as latency-critical online multiplayer games [7], [8]. These studies used a centralized Kubernetes master node to control a couple of distributed and horizontally connected Edge worker nodes to implement a DCI with a centralized control plane. Monitoring functionality native to Kubernetes aggregates cluster-wide event data, tracking CPU, memory, and network usage at worker nodes. These implementations rely on Kubernetes-native failover functionality to ensure the availability of the Edge workloads by keeping a specific replica of the application containers in the event of a workload failure. DCI implementations based on Kubernetes are insufficient to address the DCI challenges discussed in Sec. I-A. More precisely,

- First, the Kubernetes control plane can only manage a maximum of 5000 worker nodes,[5] which limits scalability. More nodes, in particular, impose more overhead on control plane functions such as across-node communication and health check collection. Apart from that, increasing the effectiveness of workload replication necessitates the addition of more worker nodes, which frequently results in a higher Total Cost of Ownership (TCO). Running two replicas on two physically separated worker nodes, for example, is more effective than running them on a single node for maintaining application HA in the event of hardware failure.
- Second, the challenge of Edge autonomy is not addressed. Kubernetes-based DCI implementations dismiss the risk of an Edge worker node being isolated from the Kubernetes control plane (e.g., a Raspberry Pi running out of batteries or disconnected from the central Kubernetes master node). As a result, all disconnected compute resources are useless until that Edge worker node rejoins the Kubernetes cluster.
- Third, despite the efforts toward federated Kubernetes clusters, recent implementations show that Kubernetes federation controllers cannot scale to a sufficient size for Edge computing use cases [9] and can lead to Edge workload deployment instability [10]. Furthermore, Kubernetes-based distributed workload deployments (e.g., multi-cluster and multi-cloud) necessitate the use of additional cluster management tools (e.g., Open Cluster Management[6]) to automate cluster registration, work distribution, and dynamic policy and workload placement.

2) OpenStack-BASED DCI IMPLEMENTATIONS

Another group of authors in [11], [12], and [13] proposed using OpenStack Availability Zones (AZs)[7] to create geographically distributed Edge computing sites that are managed centrally by the OpenStack control plane. Virtual computing, storage, and networking resources are deployed remotely on aggregated hosts in each Edge AZ site from the OpenStack central Cloud. Adding a Kubernetes cluster to each AZ's resources enables container workload orchestration and management. As a result, OpenStack-based DCI requires more hardware resources than Kubernetes-based DCI. Each OpenStack-based Edge Cloud requires a minimum of 4 CPUs and 8 GB RAM, whereas a Kubernetes-based Edge Cloud requires only 2 CPUs and 2 GB RAM. The Kubernetes control planes in the Edge sites are completely autonomous. In such cases, the OpenStack Networking service, known as Neutron, is in charge of inter-site infrastructure communications. At the same time, the Kubernetes Container Network Interface (CNI) handles container workloads' internal and external networking.

[5]https://kubernetes.io/docs/setup/best-practices/cluster-large/
[6]https://open-cluster-management.io/
[7]https://www.openstack.org/

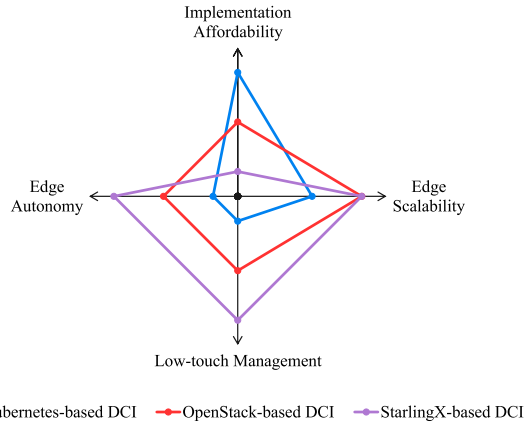


FIGURE 2. Comparison of Kubernetes, OpenStack and StarlingX DCI implementations.

3) KUBERNETES VS OpenStack-BASED DCI IMPLEMENTATIONS

While OpenStack-based Edge computing platforms are more scalable than Kubernetes-based ones; the Edge sites remain heavily reliant on the central OpenStack control plane. Assume an Edge site loses connection to the central Cloud. During the outage, no OpenStack control plane operations (such as client authentication, service discovery, and distributed multi-tenant authorization) can be performed at the impacted Edge site. Nonetheless, as long as no interaction with the OpenStack control plane is required, the Kubernetes-managed Edge workloads will be somewhat autonomous. As a result, the Kubernetes cluster can maintain some autonomy at the Edge. It is worth mentioning that some Kubernetes add-on tools, such as Virtual Kubelet,[8] can facilitate inter-cluster collaboration (e.g., offloading workloads from one Edge site to another). However, workload failover between Edge sites has not yet been addressed in OpenStack-based DCI implementations, and there is no central management of the Edge Kubernetes clusters. In conclusion, Fig. 2 depicts the benefits and drawbacks of various DCI implementations.

Our extensive research on distributed Edge infrastructure virtualization and management platforms resulted in DCI implementations based on Kubernetes or OpenStack. Both aren't fully compliant with the Edge reference architectures [3], nor are they optimized for Edge computing use cases. To the best of our knowledge, no single academic study implements a StarlingX-based DCI testbed and considers Edge workload monitoring and failover. As a result, this paper aims to provide new venues for the Edge computing research community.

B. DISTRIBUTED CONTROL PLANE DCI IMPLEMENTATIONS

1) StarlingX-BASED DCI IMPLEMENTATIONS

For the first time in academic literature in [14], where a bare metal All-In-One (AIO) StarlingX platform is used to

[8]https://github.com/virtual-kubelet/virtual-kubelet

benchmark various IoT messaging brokers deployed on an Edge computing server. Kubernetes orchestrates containerized broker applications, and Prometheus[9] records real-time metrics such as CPU usage. To abstract the IoT devices, MZBench,[10] a software tool for performing benchmark and stress tests, is installed on another node within the same Ethernet network. This StarlingX-based implementation compares the performance of various distributed IoT messaging brokers by focusing solely on the interaction between an Edge Cloud and a large number of IoT devices. As a result, it does not consider the interaction between the central and Edge Clouds.

III. StarlingX EDGE VIRTUALIZATION PLATFORM: AN INSIDE VIEW

StarlingX is a complete DCI virtualization and management platform for bare metal, virtual machines, and container-based workloads. The StarlingX platform is unique in that it combines the Linux operating system, the OpenStack Cloud platform, the Kubernetes container orchestration system, the Ceph storage platform, and the KVM virtualization solutions into a single user-friendly software stack that is ready for deployment on commercial off-the-shelf (COTS) hardware servers or virtual machines. The StarlingX platform has applications such as 5G Ultra Low-latency Communication (uRLLC) and Industrial IoT (IIoT), high bandwidth and high volume applications, and Multi-access Edge Computing (MEC) [2].

Furthermore, StarlingX can function as a fully integrated component within a use-case-oriented Edge stack, i.e., one focused on a specific Edge computing application. For example, as shown in Fig. 3, Akraino Edge Stack approved the StarlingX-based Connected Vehicle Blueprint (CVB).[11] An Akraino blueprint is a declarative configuration that defines the entire Edge stack (Cloud platform, Application Programming Interfaces (APIs), and Applications) that has been tested and validated on real hardware supported by users and community members for a specific industrial vertical (e.g., automotive, industrial automation, etc.). Akraino's CVB is an open-source MEC platform focused on Vehicle-to-everything (V2X) applications that require closed-loop information among roadside infrastructure (cameras, sensors), pedestrians, and connected vehicles, such as real-time situation awareness and high-definition maps.

StarlingX serves as an IaaS layer in Akraino CVB, virtualizing and managing the dispersed V2X Edge infrastructure, which consists of hardware servers deployed close to users in the city centre, highway, school zone, and so on. To that end, StarlingX offers a single component that integrates the key CVB dependencies, including OpenStack, Kubernetes, Open virtual Switch (OvS), and Data Plane Development Kit (DPDK), and is optimized for the V2X use case.

[9]https://prometheus.io/

[10]https://github.com/satori-com/mzbench

[11]https://wiki.akraino.org/display/AK/Automotive+Area

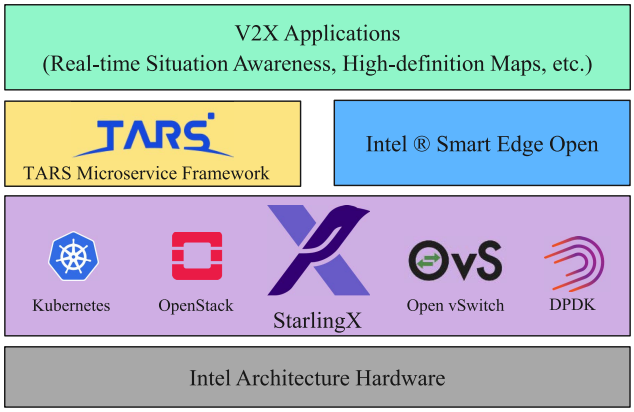


FIGURE 3. Akraino's connected vehicle blueprint.

Section III-A contains a detailed description of the StarlingX building blocks. The Platform-as-a-Service (PaaS) layer is established by the TARS[12] and Intel® Smart Edge Open[13] components. The former is a microservice framework that allows for the efficient deployment and monitoring of microservices in larger scale-out scenarios. In contrast, the latter is a collection of tailored Software Development Kits (SDKs) that provide REST-based interfaces to interact with 4G, 5G, WiFi, and O-RAN Network Functions.

A. StarlingX PLATFORM: KEY ARCHITECTURAL BUILDING BLOCKS

Because StarlingX has yet to be mentioned in academic work, we give a cohesive overview of the platform layers in Fig. 4, as supplied in version 4.0. (Aug 2020). We describe the layers from the bottom up, where the material presented has been gathered from various sources and is intended to promote the use of the StarlingX platform in academic research.

1) HARDENED LINUX

StarlingX is built on CentOS 8 Linux OS tuned for low latency and high performance and maintained with security Common Vulnerabilities and Exposures (CVE) patches.

2) OPEN-SOURCE PACKAGES

StarlingX adopted a variety of open-source software to support StarlingX Controller and Kubernetes cluster, including:

- Redfish provides a REST API for secure management of servers, storage, networking, and converged StarlingX infrastructure.
- Trusted Platform Module (TPM) provides secure storage for the private key of the StarlingX REST and web server's certificate.
- Docker provides a container runtime and local image registry for containerized services and workloads.

[12]https://github.com/TarsCloud

[13]formerly known as OpenNESS, https://github.com/smart-edge-open

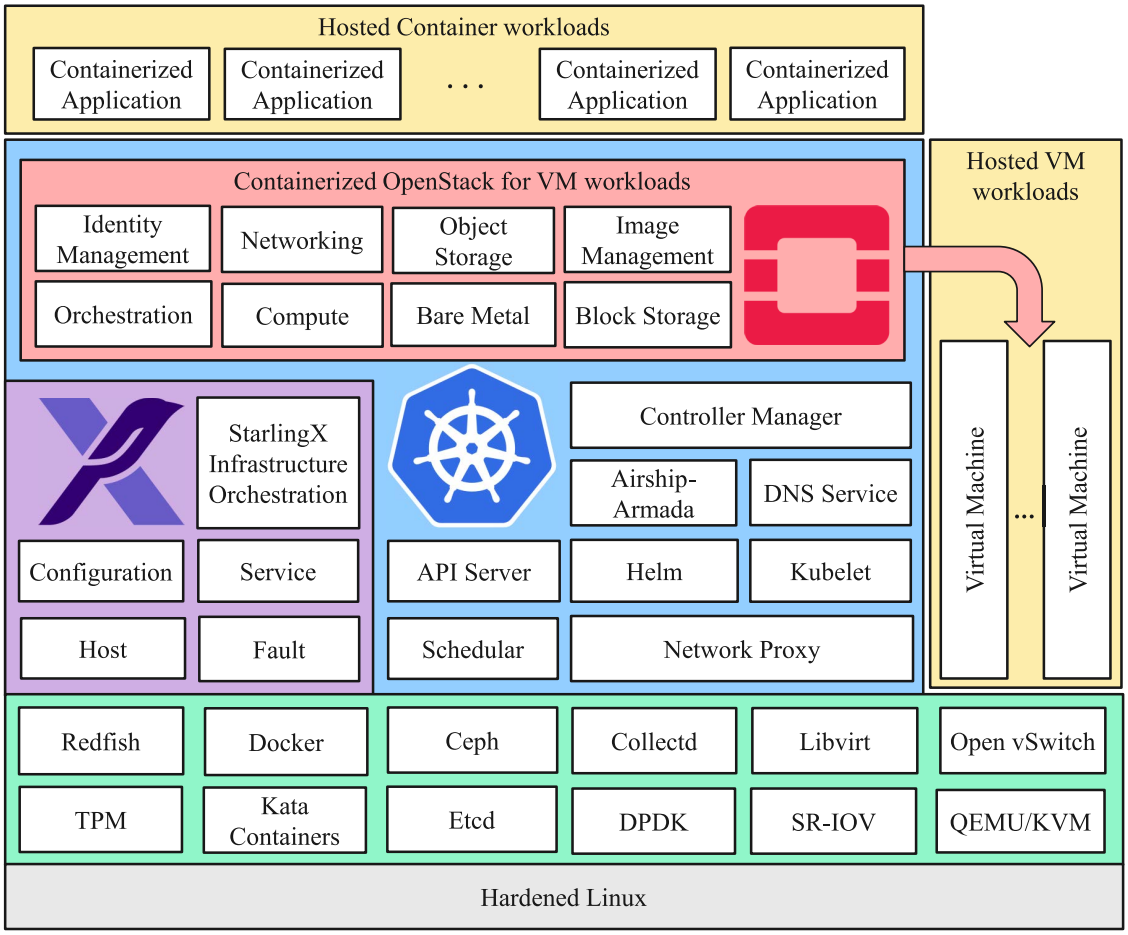


FIGURE 4. StarlingX edge virtualization platform.

- Kata Containers provides secure container runtime as an alternative to Docker and supports Time-Sensitive Networking (TSN).
- Ceph provides a software-defined storage framework for entirely distributed operation without a single point of failure.
- Etcd provides a strongly consistent, distributed and reliable key-value store for services and workloads configuration.
- Collectd provides a statistics collection daemon that periodically monitors and stores the utilization of memory, CPU, network interface, and file system.
- Single Root I/O Virtualization (SR-IOV) directly extends the physical Network Interface Controller (NIC)'s capabilities to the VM instances.
- DPDK offloads and processes network traffic on an embedded switch in SR-IOV instead of the Linux kernel.
- Libvirt provides a portable and long-term stable C API for managing various operating system virtualization technologies.
- OvS provides a distributed virtual multilayer stack that enables switching for the hardware virtualization environments.
- Quick EMUlator (QEMU) takes advantage of the Kernel-based Virtual Machine (KVM) acceleration to benefit the host and the VM workloads.

3) KUBERNETES

StarlingX leveraged Kubernetes to become the industry's de facto container orchestration technology to provide automated workloads deployment, scaling, and management. In Kubernetes, one or more containers are relatively tightly coupled in one *Pod*, sharing the same storage, memory, CPU and network resources. Pods are the smallest deployable units of containerized workloads that can be created and managed in Kubernetes. Below are the main components of Kubernetes and its additional deployment automation tools.

- API Server is the front end of the Kubernetes control plane that validates and configures data for the API objects such as Pods. It services REST operations and exposes a Kubernetes API to the cluster's shared state through which end users, different parts of the Kubernetes cluster, and external components communicate.
- Controller Manager provides a daemon that embeds a couple of non-terminating control loops, aka *Kubernetes Controllers*, that watch the shared state of the

Kubernetes cluster through the API Server and make changes attempting to move the current cluster state towards the desired state. Examples of Kubernetes controllers are the Replication Controller, Endpoints Controller, Name-space Controller, and Service Accounts Controller.

- Scheduler provides the placement decision of each newly created Pod. It determines the optimal node for each Pod to run on by filtering the feasible nodes that meet the Pod-specific scheduling requirements and scoring them according to individual and collective resource requirements. e.g., hardware, software and policy constraints; affinity and anti-affinity specifications, data locality, inter-workload interference. The scheduler assigns the Pods to the worker node with the highest-ranking score.
- Kubelet is the Kubernetes agent running on every node of the Kubernetes cluster to register the node with the API Server and ensure that the containers are healthy and running as described in the specification of their Pod deployment manifest.
- Helm is a tool to automate the deployment of complex Kubernetes workloads using a packaging format called charts and parameterizing the deployment configurations. A Helm Chart is a collection of files that describe a related set of Kubernetes resources. The deployment configurations of a single chart, which declares multiple Kubernetes resources, can be centrally stored and managed through Helm, abstracting the complexity of manual changes. In other words, *Helm* is a package manager for Kubernetes.
- Airship-Armada is an orchestrator of multiple Helm Charts. It manages multiple Helm Charts with dependencies by centralizing all configurations in a single Armada YAML arranging Helm Charts into Chart Groups and providing life-cycle hooks for all Helm releases.
- Domain Name System (DNS) provides an IP resolution service to let Kubernetes components access the Internet on the Operations, Administration and Maintenance (OAM). E.g., a remote Docker registry, a Network Time Protocol (NTP) server, or a Precision Time Protocol (PTP) server, when needed. It is noteworthy that StarlingX can operate without a configured DNS service. However, a DNS service should be in place to ensure that links to external references work as expected.
- OAM Network Proxy lets the platform components and workloads running on StarlingX reach the public network, e.g. remote Docker registries, when StarlingX-DCI is behind a corporate firewall or proxy.

4) StarlingX SERVICES

To bridge the gaps in the open-source ecosystem, StarlingX introduced the following components running on bare metal, aka Flock services, to enhance the platform deployment, maintainability and operations.

- Configuration Management Service provides a single pane of glass to manage the inventory of resources (e.g., CPUs, GPUs, memory, storage, crypto/compression hardware, huge pages) at Edge sites. It supports auto-discovery and installation of new nodes, which is helpful in the case of a geographically distributed large number of remote Edge sites. It integrates with OpenStack Dashboard Graphical User Interface (GUI) and Command Line Interface (CLI).
- Host Management Service provides life-cycle management functionality for the dedicated bare metal Edge servers hosting the StarlingX platform. This tool is vendor-agnostic and can detect host failures and initiate automatic recovery by monitoring and alarming for cluster connectivity, critical process failures, resource utilization thresholds, and hardware faults via REST API.
- Fault Management Service provides a tool to set, clear and query custom alarms and logs for significant events for infrastructure StarlingX nodes, VMs, and other virtual resources. The Active Alarm List and Active Alarm Counts Banner are available through OpenStack Dashboard GUI.
- Service Management Component provides HA for cluster management services through redundancy models such as N+M or N across multiple nodes. It also supports using multiple messaging paths to avoid split-brain communication failures and active or passive monitoring and to specify the impact of a service failure with an entirely data-driven architecture.
- Software Management service provides an automated framework to update the software in all layers of the StarlingX platform - CentOS, open-source packages, StarlingX, Kubernetes, or OpenStack components for corrective content, security, or new functionality. It supports rolling upgrades, including parallelization and live migration across the nodes, i.e., host reboot with moving workload off the node. This Software Management framework is accessible via OpenStack Dashboard GUI, CLI, or REST API.
- Infrastructure Orchestration. This component is built based on OpenStack Regions, belongs to the central Cloud, and handles system-wide infrastructure orchestration functions such as deployment and management of Edge Clouds, a portal for shared configuration across all Edge Clouds, fault aggregation, and patching orchestration. Edge Clouds communicate with the central Cloud through strictly REST APIs over an L3 network and run the control plane functions for autonomous operation.

5) CONTAINERIZED OpenStack SERVICES

StarlingX leverages different building blocks from the OpenStack platform services to strengthen its platform and support VM workloads. The leveraged OpenStack services are containerized and deployed on top of Kubernetes using

OpenStack Helm Charts. Below is a description of the OpenStack building blocks:

- Identity Management Service, aka Keystone, provides authentication and authorization services throughout the Kubernetes cluster.
- Orchestration, aka Heat, service automatically deploys VM instances, networks, and other OpenStack services.
- Networking Service, aka Neutron, provides various networking services to VM workloads such as IP address management, DNS, Dynamic Host Configuration Protocol (DHCP), load balancing, and security groups (e.g., network access rules like firewall policies).
- Bare Metal Service, aka Ironic, supports bare-metal machine provisioning (as opposed to virtual) through standard and vendor-specific remote management protocols. It also provides the Compute service with an interface to manage the physical servers as virtual machines.
- Compute Service, aka Nova, supports provisioning and managing to compute instances, aka virtual servers. Through Bare Metal Service, Compute Service provisions bare metal servers.
- Image Management Service, aka Glance, provides disk-image management services, including image discovery, registration, and delivery services to the Compute service.
- Block Storage Service, aka Cinder, provides compute instances an access and life cycle management of persistent storage devices from creating and attaching volumes to instances to their release.
- Object Storage Service, aka Swift, provides resilient data storing and retrieving services such as media files, VM images, and backup files.

6) StarlingX FAULT MANAGEMENT SERVICE - STATUS UPDATE

The StarlingX fault management service is a complete implementation of the ETSI MEC Fault Management interface [15], clause 7.3.1, which specifies the platform's alerting and logging features. This solution focuses on the platform's overall health, such as exceeding CPU, memory, or file system thresholds, and records of maintenance events such as found, locked, reset, rebooted, deleted, disabled, enabled, or powered-off/on hosts. Because workloads are outside the platform-managed infrastructure resources, the health of deployed workloads (e.g., application containers) is not considered by the StarlingX fault management service.

When it comes to crucial IoT applications placed on Edge sites with intermittent connectivity to the central Cloud, however, reacting to events created by workloads or the infrastructure is critical to ensuring application uptime. Indeed, both platform and workload flaws can impair the quality of service of IoT applications and, as such, should be handled similarly. For example, when a workload fails, or an Edge computing node crashes, an alarm of such a catastrophic event is required to initiate a recovery procedure. According to a recent Internet proposal from the IETF [5], the OAM of an Edge computing platform should consider the computing environments (e.g., software containers) in platform management tasks such as monitoring and diagnostics of failures and recovery measures. In Sec. V-C, we propose a failover feature to be part of the VNF-manager of the conventional NFV-like management and orchestration paradigm [16], resulting in an Edge computing platform that is aware of workload failures.

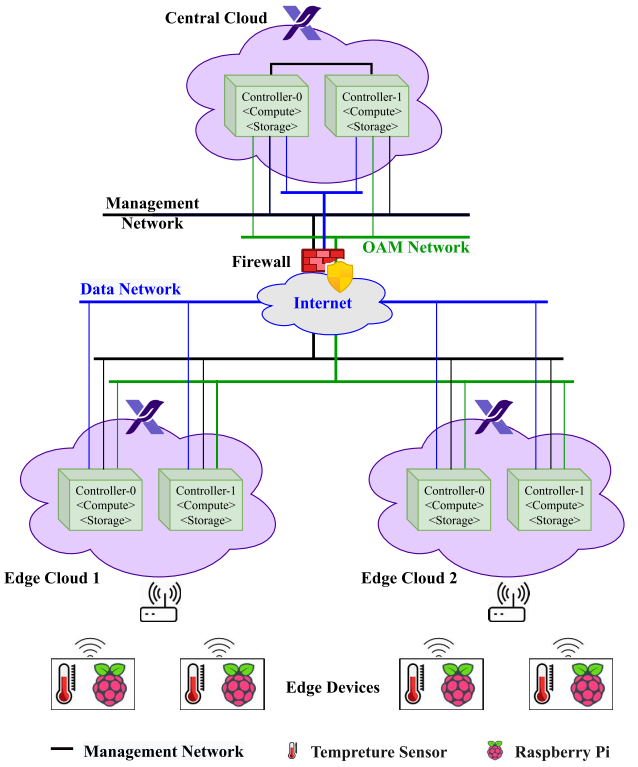


FIGURE 5. StarlingX-based DCI testbed architecture.

IV. StarlingX-BASED DCI TESTBED IMPLEMENTATION

A. TESTBED ARCHITECTURE

As shown in Fig. 5, our DCI testbed is comprised of three tiers of Cloud computing nodes. We extended the StarlingX distributed Cloud installation guidelines to run on VMs [4], resulting in a cost-effective DCI testbed implementation for the research community. In [17], we documented the installation instructions for a virtual StarlingX-based DCI testbed, where the implementation process begins with provisioning the central Cloud, followed by installing the Edge Clouds (also known as Near Edge), and finally attaching the Edge devices (also known as Far Edge). For the central and two Edge Clouds, we used the StarlingX All-In-One Duplex (AIO-DX) deployment configurations,[14] which include a pair of HA servers for the controller, compute, and storage Cloud functions. The controller function runs HA services across the two servers in either Active/Active or Active/Standby mode.

The central Cloud is hosted in the Carleton University data centre. The two Edge Clouds are hosted on private home

[14] https://docs.starlingx.io/deploy/index.html

TABLE 1. StarlingX-based DCI testbed BoQ.

System Summary	Hardware Specification per bare metal server	Recommended Quantity per testbed
Processor[p]	8 Cores/socket	3
Memory[m]	64 GB	6
Storage[s]	500 GB SSD	3
Network card[n]	Management: 10GE OAM: 1GE Data: 10GE	3
Edge device[ed]	Raspberry Pi 4 Model B	4
Edge Sensor[es]	Temperature Sensor	4
Servers' enabled BIOS settings	• Hyper-Threading Technology • Virtualization Technology for Directed I/O (VT-d) • Virtualization Technology for Connectivity (VT-c) • Intel® Virtualization Technology (VT-x)	

[p] Dual-CPU Intel® Xeon® Processor E5 26xx Family or Single-CPU Intel®Xeon®D-15xx Family
[m] 32GB PC4-19200 2400MHz DDR4 ECC Registered DIMM
[s] Intel® SSD D3-S4610 Series, 2.5in SATA 6 Gb/s, 3D2, TLC
[n] Intel® Ethernet Converged Network Adapter XL710-QDA1 10/40 GbE
[ed] BCM2711, 8GB LPDDR4-3200 SDRAM
[es] DHT22 AM2302, Working voltage: DC 5V

servers, with two L3 VPN tunnels configured on the University Firewall. The StarlingX System Controller is housed in the central Cloud and is in charge of managing and orchestrating the Edge Clouds, synchronizing select configuration data across all Edge Clouds, and monitoring Edge Cloud operations and alarms. For OAM and management, the central Cloud connects each Edge Cloud via two separate L3 networks, with the networks' gateways configured on the central Cloud System Controller. The OAM network connects the System Controller to the Edge Clouds. In contrast, the management network connects to platform-level management services. The data network connects application workloads and allows access to public networks such as the Internet. We built the data network on the physical network using the Virtual Local Area Network (VLAN) identifier, allowing multiple virtual data networks to exist on the same physical network. We used Raspberry Pi as an Edge device linked to the Edge Clouds via the same LAN. Each Raspberry Pi is flashed with the Raspbian operating system and configured by installing Docker. It is worth noting that the Edge devices are not managed by the StarlingX platform or Kubernetes but are manually configured to connect the Edge Clouds VMs. The recommended hardware specifications and the Bill of Quantities (BoQ) of the used equipment are shown in Table 1.

B. MICROSERVICE-ARCHITECTED TEMPERATURE MONITORING APPLICATION

We created a microservice-architected temperature monitoring application to validate our StarlingX DCI testbed installation. As shown in Fig. 6, the application is deployed across all levels of the DCI (central Cloud, Edge Clouds, and Edge devices) to serve as a sample multi-container workload. The application aims to record temperature sensor readings from geographically distributed Edge Clouds centrally. This

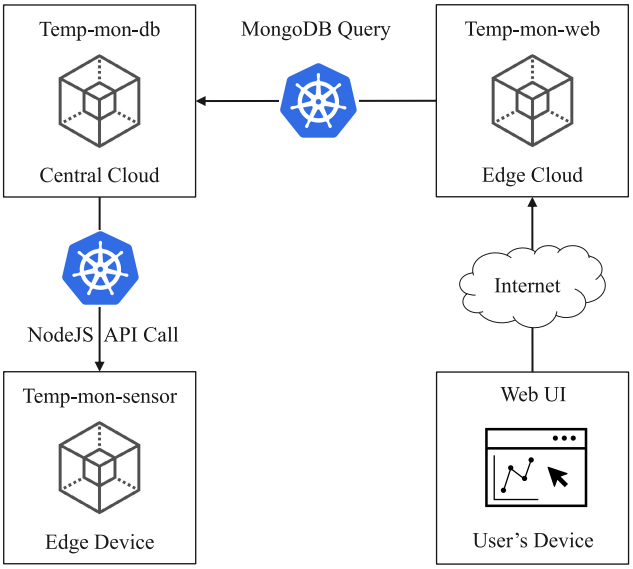


FIGURE 6. Temperature monitoring application architecture.

temperature monitoring application is useful for various Edge computing use cases, including smart agriculture, industrial automation, public safety, and optimized cyber-physical systems (e.g., smart buildings). The application is made up of the three containers listed below:

- Temp-mon-sensor: This container runs on Raspberry Pis and collects temperature readings from a physical temperature sensor before reporting them via a Node.js API. This container responds to any HTTP GET request with the current temperature readings detected by the sensor.
- Temp-mon-db: A MongoDB instance with a persistent storage backend runs on this container's central Cloud. It queries the Temp-mon-sensor container regularly to obtain the most recent temperature reading and stores it in the database with a timestamp indicating the temperature recorded time.
- Temp-mon-web: This container runs on the Edge Clouds and provides a user interface for monitoring current and historical temperature readings. The frontend's temperature graph in Fig. 7 obtains temperature data by querying the Temp-mon-db container using a number of filters, including the date range and the number of desired results. This container represents a real-world Edge workload that runs on the central Cloud and the Edge devices.

V. PROPOSED EDGE FAILOVER MECHANISM

This section explains the RTO's structure before introducing our dynamic monitoring and failover mechanism.

A. RECOVERY TIME OBJECTIVE

The centralized monitoring service should detect it when an Edge workload or node in a DCI fails. The failover procedure is then initiated to carry out a predefined recovery process.

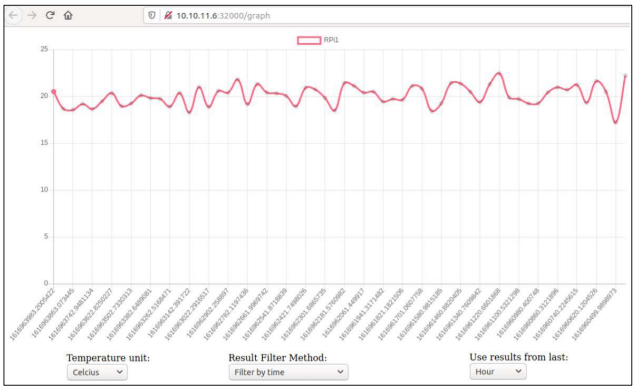


FIGURE 7. Temperature graph as monitored from *Temp-mon-web* container.

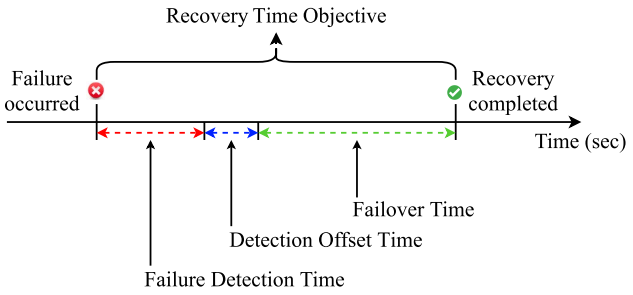


FIGURE 8. Breakdown of the recovery time objective.

The downtime that occurs between the time the failure occurs and the completion of the recovery is referred to as the Recovery Time Objective (RTO). A low RTO is always recommended because it is one of the key parameters determining the total cost of failure. Fig. 8 depicts the breakdown of RTO into the three time periods:

- Failure detection time (T_d): The time between when a failure occurs and when the monitoring service detects it. In a DCI-like environment, this time includes the propagation delay between the Edge site where the failure occurred and the centralised monitoring service at the central Cloud.
- Detection offset time (T_o): The time required to ensure the failure has occurred. This offset aids in the dismissal of false-positive failures (FPF) alarms. Before implementing the failover process, which may involve moving workloads from one Edge node to another, we should proceed with caution and request a couple of health checks from the affected Edge site, or wait until more statistics confirming the failure status arrive at the central Cloud. Sec. V-B goes over the various monitoring service implementation options in depth.
- Failover time (T_f): The time it takes to implement the recovery process and return to the state it was in before the failure occurred. Monitoring the newly deployed Edge workload or the installed node until stable conditions are reached is part of the recovery process. Sec. V-C describes our proposed failover functionality.

$$RTO = T_d + T_o + T_f \tag{1}$$

B. POLLING-BASED VS. EVENT-DRIVEN MONITORING

The StarlingX fault management service offers a centralised tool for configuring, clearing, and querying custom alarms and logs for significant events on infrastructure StarlingX nodes, VMs, and other virtual resources. Similarly, we implemented our proposed monitoring and failover functionality on StarlingX System Controller in the central Cloud to operate autonomously and communicate with the geographically dispersed Edge Clouds via a REST API over L3 networks. Before delving into the implementation and experimentation details, we classified and compared the ecosystem of open-source monitoring solutions. Following is a summary of our findings and observations on the current monitoring solutions.

Existing workload monitoring solutions are divided into two groups based on the architectural design of statistics collection: polling-based monitoring such as Prometheus[15] and Kubernetes-Nagios,[16] and event-driven monitoring such as ELK[17] Stack, Zabbix,[18] WeaveScope,[19] and Jaeger.[20] The Edge workloads are added to a monitoring list for the former, and a regularly scheduled poll process - hence the name - will run in the central Cloud to collect the status of each workload. For the latter, monitoring agents are deployed in the Edge Clouds to actively report the gathered events about the target workload to the central Cloud (hence the name).

The trade-off between polling-based and event-driven monitoring in a DCI is complex. As a result, in the following two paragraphs, we present the figures of merit for comparing polling-based and event-based failover mechanisms.

1) PERSPECTIVE 1 - COST

Polling-based monitoring is far easier to implement than event-based monitoring because it does not require frontend agents at Edge nodes and can operate with low overhead on Edge resources (i.e., compute, memory, and storage). However, as requests and responses bounce between the central and Edge clouds, such monitoring generates extra traffic. The more workloads in the monitoring list, the more overhead traffic the network will experience. In practise, the overhead monitoring traffic can be reduced by tuning the time

[15] https://prometheus.io/
[16] https://github.com/colebrooke/kubernetes-nagios
[17] ELK is the acronym for four open-source projects: ElasticSearch, a search and analytics engine that stores Kubernetes logs; Logstash, a log aggregator that captures and processes logs before shipping them to Elasticsearch; Kibana, a reporting and visualization tool; and Beats, a lightweight data shippers used to send logs and metrics to Elasticsearch, either directly or via Logstash.
[18] https://www.zabbix.com/
[19] https://github.com/weaveworks/scope
[20] https://www.jaegertracing.io/

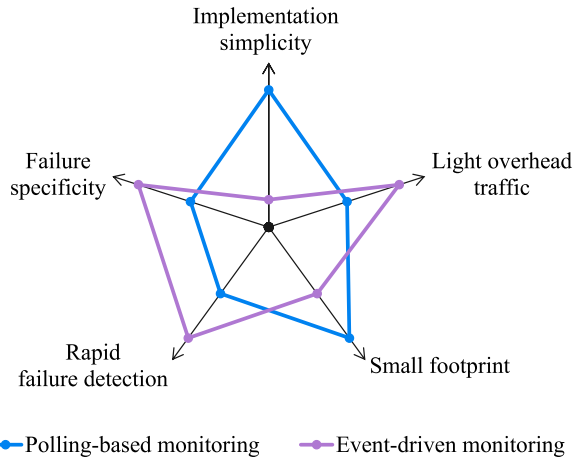


FIGURE 9. Polling-based vs. event-driven monitoring services.

window between sending two consecutive statistics collection requests using a timing variable such as `polling_period`.

Event-based monitoring, on the other hand, can operate with minimal overhead traffic. That is, when agents report events on an ad hoc basis, for example, whenever a failure occurs, or when they periodically report a summary of workload health statistics, without being prompted by the central Cloud monitoring process.

2) PERSPECTIVE 2 - QUALITY

For polling-based monitoring, failure detection requires a minimum delay of one Round-Trip Time (RTT) — for a request and a reply — while half of this delay is needed for event-based monitoring. Failure specificity is proportional to the rate of FPFs. When there are few FPFs, the failure specificity is high, and vice versa. Obtaining FPFs is a given in Cloud environments due to the ephemeral nature of workloads and the virtualized infrastructure resources. Because of the two-way communication between the central and Edge clouds, polling-based monitoring is more likely to receive FPFs. For example, if the link delay suddenly exceeds the monitoring `timeout`, no replied health check arrives for a period of time. In the case of event-based monitoring, however, such a spike in delay will result in a longer failure detection time rather than FPF. Fig. 9 depicts the advantages and disadvantages of polling-based and event-driven monitoring in a DCI.

C. MONITORING AND FAILOVER FUNCTIONALITY

We created a simple polling-based monitoring and failover functionality for this study that is always active on the StarlingX System Controller in the central Cloud and polls the health checks of the relevant container workloads from the Kubernetes API servers in both Edge Clouds. The monitoring and failover functionality, as shown in Fig. 10 starts by initializing the `monitored_workloads`, which are the Edge workloads to be monitored; the `polling_period`, at which the health checks are collected; the `timeout`,

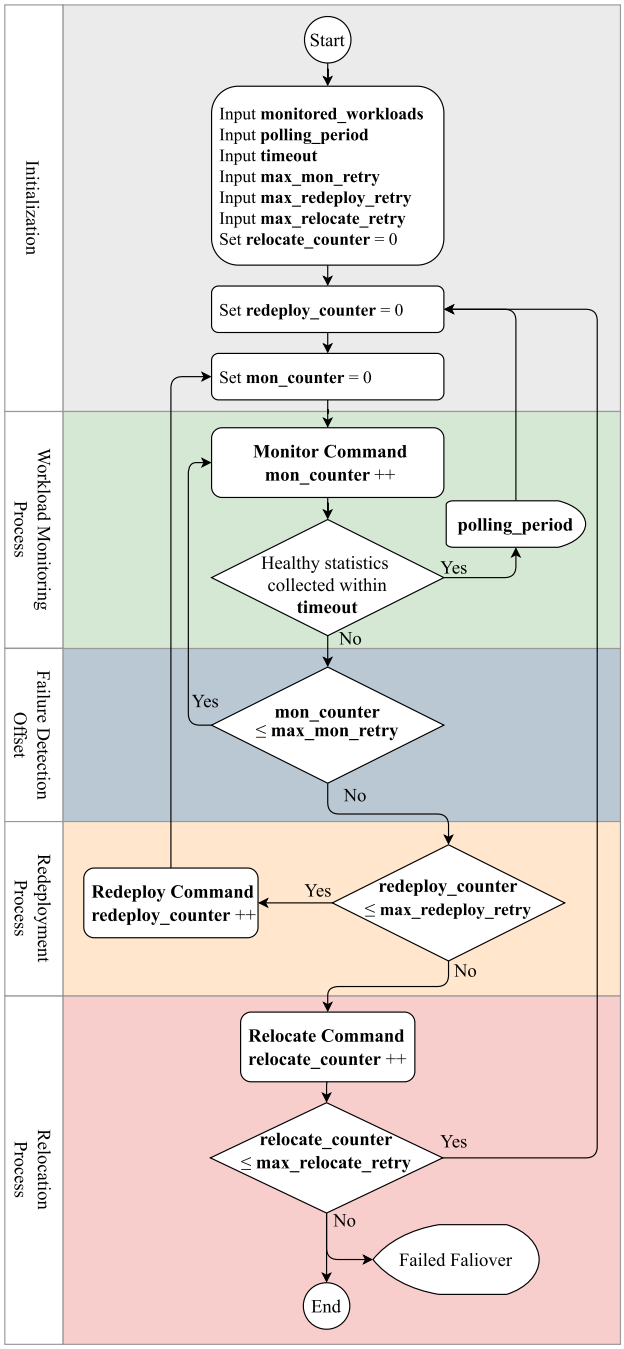


FIGURE 10. Proposed monitoring and failover process flowchart.

which is a countdown timer to receive the health checks; `max_mon_retry`, which defines the maximum retries of running the `Monitor Command` before declaring a true failure is detected; `max_redeploy_retry`, which set the maximum retries of redeploying the failed workloads before relocating them to another Edge or central Cloud; `max_relocate_retry`, which is the maximum number of retries of relocating the failed workloads before declaring the failed failover process; `relocate_counter = 0`, which reset the number of relocation attempts to zero.

The workload monitoring process keeps executing the `Monitor Command`, which polls the health checks of the relevant container workloads every `polling_period`. The number of times the `Monitor Command` is called is tracked by the `mon_counter`. In our implementation, the `Monitor Command` lists the Helm Charts active on a given node and compares them against the list of Helm Charts that should be active. When any discrepancies are detected, or the monitoring process fails to collect the health checks within a `timeout`, a workload failure is detected. As such, the upper limit of the failure detection time, T_d, is given by Eq. 2. That is when the failure happens immediately upon receiving a healthy workload status.

$$T_d \leq \texttt{polling\_period} + T_{MC} + \texttt{timeout}, \quad (2)$$

where T_{MC} is the `Monitor Command` execution time.

Before embarking on the redeployment and relocation recovery processes, our proposed failover functionality allows for `max_mon_retry` times repeating the `Monitor Command` and waiting for a maximum duration of `timeout` for each call of the `Monitor Command` to collect the health statistics of the relevant container workloads. The maximum elapsed time during this step is given by Eq. 3, which is the Detection Offset time, T_o. If the received health statistics indicate that the relevant workloads are healthy, then the workload monitoring process has detected an FPF, and no need to implement the recovery commands. In this case, the failover process will return to the monitoring mode as described above.

$$T_o \leq \texttt{max\_mon\_retry} \times (T_{MC} + \texttt{timeout}) \quad (3)$$

If the monitoring process does not receive any health statistics during the Detection Offset time (or received but the relevant workloads were missing), the failover process will trigger the `Redeploy Command` as a first corrective measure, assuming the Edge node is accessible and healthy, but relevant workloads are not. In our implementation, the `Redeploy Command` executes a redeployment of the Helm Chart of the relevant container workloads. The failover process allows to repeat the `Redeploy Command` for `max_redeploy_retry` times before triggering the second corrective measure, `Relocate Command`. As such, the maximum workload failover time is given as follows:

$$T_f(redeploy) \leq \texttt{max\_redeploy\_retry} \times (T_{DC} + T_o), \quad (4)$$

where T_{DC} is the `Redeploy Command` execution time. Notice that after the `Redeploy Command` the failover process repeats the `Monitor Command` for `max_mon_retry` times to verify whether the redeployment succeeds to bring the relevant workloads up.

If the Edge node is not reachable or fails, the failover process triggers the `Relocate Command` to relocate the entire workloads from the failed Edge node to another healthy one. In our implementation, we chose to place the relocated workloads on the central Cloud. However, in general, this choice can be subject to a workload placement process, which is beyond the scope of this study. The failover process allows for a maximum of `max_relocate_retry` relocation trials before it exists with a failed failover status. Note that for each relocation trial, the redeployment process can take up `max_redeploy_retry` times if the relocation fails. As such, the maximum relocation time is given as follows:

$$T_f(relocate) \leq \texttt{max\_relocate\_retry} \times (T_{LC} + T_f(redeploy)), \quad (5)$$

where T_{LC} is the `Relocate Command` execution time.

The polling-based monitoring and failover functionality developed is completely modular. It works by creating Edge Cloud objects and assigning each one a set of triggers that specify which events to capture and how to respond to them. It is worth noting that our proposed failover functionality is built entirely outside of the context of the StarlingX, relying solely on Helm and Kubernetes and avoiding any contact with the StarlingX platform. We made all study artefacts available on GitHub[21] in order to bring the research community up to speed on testbed implementation and Edge failover measurement collection, and to enable reproducible results with the least amount of effort. We specifically provide the following deliverables:

- The installation guidelines for a three-tier DCI.
- A microservice-architected IoT application for collecting temperature readings from Raspberry Pi-controlled temperature sensors.
- The Helm charts are used to deploy the aforementioned microservices on the DCI.
- A polling-based monitoring and failover capability for recovering from Edge workload or node failures.

VI. EXPERIMENTS AND MEASUREMENTS

The proposed failover functionality aims to restore connectivity to failed or inaccessible workloads as soon as possible, ensuring that the failover occurs in a timely manner. To accomplish this, we conducted two experiments on the DCI testbed described in Sec. IV: container workload failure (Experiment 1) and Edge node failure (Experiment 2), in order to determine the optimal values of the failover timing parameters `polling_period` and `timeout` that result in the shortest RTO. We purposefully caused the errors described in the two test experiments by either shutting down a container or an entire Edge Cloud. For experiments 1 and 2, the corrective measures are a redeployment of the relevant container on the same Edge node and a relocation to the central Cloud, respectively. When an Edge Cloud becomes unreachable to the central Cloud, for example, due to hardware failures or networking issues, relocating the Edge workloads to another Edge Cloud that is accessible to both IoT devices and the central Cloud will reduce the

[21] https://github.com/MohammedAbuibaid/Stx_DCI_Failover

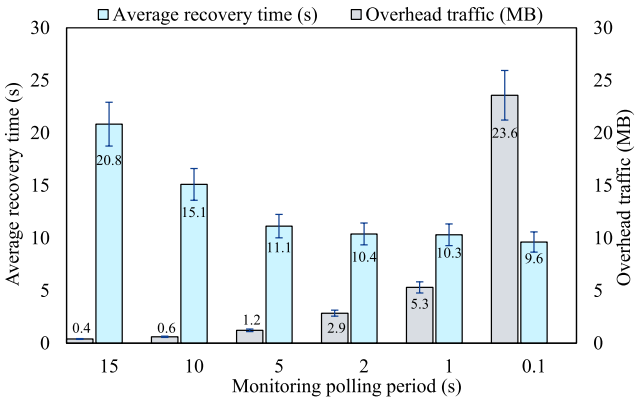


FIGURE 11. Experiment 1: Effect of `polling_period` on the average recovery time.

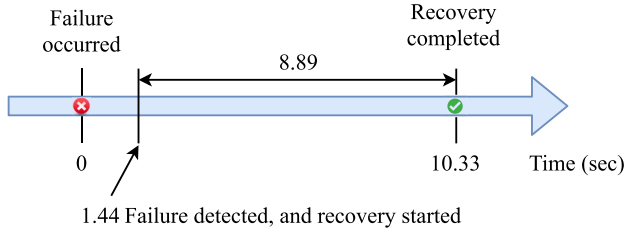


FIGURE 12. Experiment 1: Breakdown of the recovery time objective.

RTO. Experiment 2 assumes that the IoT device has no hardware failures and that it can communicate with another healthy Edge Cloud or the central Cloud. We repeated each experiment ten times and calculated the measurements with a 95% confidence interval. The other failover parameters used in both experiments are shown in Table 2.

A. EXPERIMENT 1: CONTAINER WORKLOAD FAILURE

This experiment investigates the effect of shortening the `polling_period` on the RTO and monitoring overhead traffic. In Fig. 11, we can see the downward trend of the average recovery time as the `polling_period` is decreased, approaching a plateau at around 9 seconds. To understand this trend better, we need to look at the various components that add up the total recovery time. As shown in Fig. 12, the RTO is the summation of two values: Failure Detection Time, T_d, and Failover Time, $T_f(redeploy)$, i.e., container startup time. Notice that the `max_mon_retry` value is set to 0, to nullify the Detection Offset Time, T_o, which we consider in Experiment 2. Numerically, the RTO of this experiments is calculated using Eq. 2 and Eq. 4 as follows:

$$\begin{aligned} RTO &= T_d + T_f(redeploy) \\ &= 1.44 + 8.89 \\ &= 10.33 \quad \text{seconds} \end{aligned}$$

The `Monitor Command` consumes an average delay of 1.44 seconds for failure detection, whereas the average container startup time is 8.89 seconds. That is the failover time to redeploy the failed workload. From the RTO breakdown measurements, it is noticeable that the failure detection time is the main contributor to the decaying recovery time, in Fig. 11. In contrast, the workload startup time remains relatively constant and out of our control (unless using a faster CPU, adding more RAM, or a faster link connection between the centralized failover process and the Edge node of the relevant workload). We can infer that when the `polling_period` values go lower than 5 seconds, we begin to see diminishing returns in the recovery time. However, the gain of lowering the RTO comes at the cost of generating more monitoring overhead traffic, as shown on the secondary axis of Fig. 11. Such trade-off shows that a `polling_period` of 5 seconds is the most reasonable point to configure the monitoring functionality.

TABLE 2. Failover experimentation parameters.

Parameter	Value	
	Experiment 1	Experiment 2
`polling_period` (sec)	15, 5, 10, 2, 1, 0.01	1
`timeout` (sec)	1	5, 3, 1, 0.1
`max_mon_retry`	0	3
`max_redeploy_retry`	1	
`max_relocate_retry`	1	
`monitored_workloads`	Temp-mon-web	

To quantify the overhead traffic, we captured the total amount of monitoring requests and replies between the central and Edge Clouds during a 60 seconds time window as shown in Table 3. The relevant Edge workloads are left intact and healthy to get the maximum possible overhead traffic and avoid entering the failover process, wherein the `Redeploy Command` would consume some time and thus generate fewer queries. The relation between the actual number of queries and the generated monitoring overhead is fairly linear with a 100 KB per query. It is worth noting that when the `polling_period` drops below 5 seconds, the number of actual queries starts to divert from the expected theoretical values. The reason is that the monitoring service consists of two processes: the `Monitor Command` process, which approximately consumes 0.16 seconds on the central Cloud server; and the sleep process, which is the `polling_period` value. For example, when the `polling_period` is 0.01 seconds, the monitoring process can generate a maximum of $\lfloor \frac{60}{0.16+0.01} \rfloor = 230$ queries.

B. EXPERIMENT 2: EDGE CLOUD FAILURE

To investigate the effect of the `timeout` value on the average workloads relocation time after an Edge Cloud failure, we considered a maximum of 1 failed `Redeploy Command`'s is enough to declare an Edge Cloud is unreachable, i.e., `max_redeploy_retry` = 1 and to trigger the `Relocate Command`. The `polling_period` is set as low as 1 second to examine the possibility of getting FPFs. Fig. 13 shows a downward trend of the average

TABLE 3. Experiment 1: Monitoring overhead vs. Polling period.

Polling period (seconds)	No. Queries (every 60 seconds)		Actual Overhead Traffic (MB)
	Theoretical	Actual	
60	1	1	0.1
30	2	2	0.2
20	3	3	0.3
15	4	4	0.4
10	6	6	0.6
5	12	12	1.2
2	30	28	2.9
1	60	52	5.3
0.01	600	230	23.6

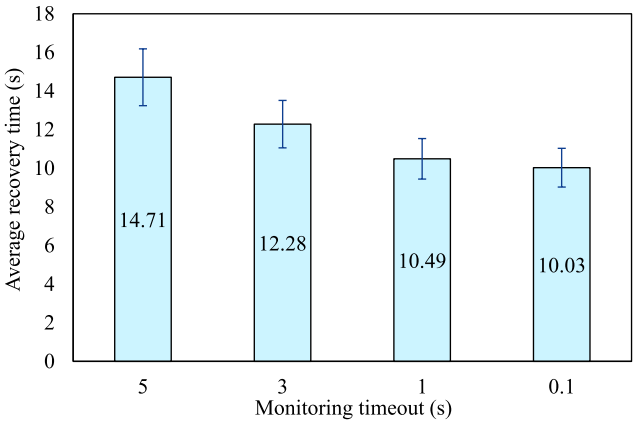


FIGURE 13. Experiment 2: Effect of `timeout` value on average recovery time.

recovery time as the `timeout` value decreases. As such, keeping a low `timeout` value is of high importance to lower the RTO. However, if the `timeout` value is too low, the `Monitor Command` will time out before receiving a valid response, resulting in an FPF. A FPF will cause the detection loop to restart, resulting in an additional incurred delay up to `max_mon_retry` × `timeout`. Therefore, we should avoid FPFs at all costs. Because the nature of our testing environment allows for our nodes to be directly connected, a `timeout` value of 0.1 second was found to be stable. However, if nodes are remote or travelling over slow networks, most likely a higher `timeout` value would be needed to avoid FPFs.

The timeline in Fig. 14 shows the breakdown of the average RTO from the moment the failure occurs till the recovery gets completed. The failure detection time is the same as Experiment 1 since the `polling_period` does not change. However, the polling `timeout` introduces an additional delay of 0.3 seconds, the `timeout` value multiplied by the number of timeout `max_mon_retry`. Such delays are inevitable because of the polling-based monitoring process, which constantly polls each Edge Cloud to determine its workload health status. In contrast, these delays are approximately eliminated in event-based detection, wherein failures

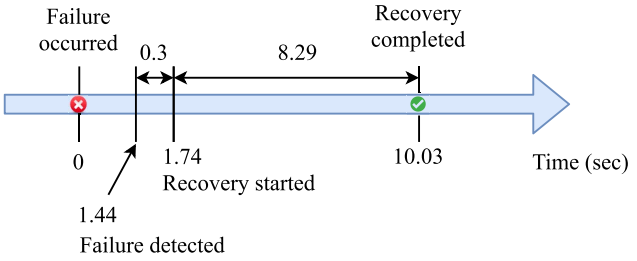


FIGURE 14. Experiment 2: Breakdown of the recovery time objective.

are nearly instantaneously detected and propagated to the central Cloud. Then, remedial actions can immediately occur. The average recovery time is 8.29 seconds, which is slightly smaller than the recovery time of Experiment 1 because the `Relocate Command` is configured to resettle relevant Edge workloads to the central Cloud, where the failover process runs. Numerically, the RTO of this experiments is calculated using Eq. 2, Eq. 3 and Eq. 5 as follows:

$$\begin{aligned} RTO &= T_d + T_o + T_f(relocate) \\ &= 1.44 + 0.3 + 8.29 \\ &= 10.03 \quad \text{seconds} \end{aligned}$$

To be fair, we have limited our evaluation to the StarlingX-based DCI implementation for the following reasons:

- First, due to the lack of StarlingX's competitors, conducting a benchmark study at this time may be impossible. The Akraino Edge Stack[22] (an open-source Linux Foundation-led project), for example, has an approved StarlingX-based Far Edge Distributed Cloud Blueprint[23]; however, progress has stalled since February 2020. As a result, benchmarking comparisons are difficult to make.
- Second, because our evaluation is based on a real-world testbed implementation, we are constrained by the available resources (number of servers, CPU, memory, etc.). As a result, large-scale experiments are difficult.
- Finally, we continue to believe that the current testbed evaluation (even if it is in early stage) is an important contribution to the deployment of the Edge infrastructure virtualization platforms.

VII. CONCLUSION AND FUTURE WORK

Although a DCI can address low-latency, bandwidth bottleneck, and data security and privacy issues, it faces significant challenges in managing Edge workloads and Clouds. StarlingX, an open-source software stack for Edge infrastructure management, uses OpenStack, Kubernetes, and other open-source building blocks to implement a distributed control plane DCI. Throughout this study, we introduced the

[22]https://www.lfedge.org/projects/akraino/
[23]https://wiki.akraino.org/display/AK/StarlingX+Far+Edge+Distributed+Cloud

academic research community to the StarlingX platform for the first time. Then, we built a DCI testbed on StarlingX and proposed monitoring and failover functionality for Edge workloads. We conducted experiments to investigate the relationships between workload redeployment and relocation recovery times to the failover timing variables (`polling_period`, and `timeout`) in order to develop an optimal failover process. In terms of societal contributions, our proposed failover scheme can maintain high availability Edge workloads. For example, IoT applications with reliability constraints can benefit from our work by reducing end-user downtime.

To be clear, our proposed failover functionality is not event-driven, but it constantly polls each Edge node and workload to determine its health status, which introduces an additional delay in the form of detection time. Although polling is slightly less efficient, the detection delay adds only 2-3 seconds to the total recovery time and is simple to implement.

For future work, we recommend that the research and StarlingX communities consider the following directions:

- Extend the StarlingX Fault Management Service to include workload monitoring and failover capabilities, as well as event-driven triggering capabilities similar to the open-source ELK suite. Webhooks provide an event-driven method of executing arbitrary code, allowing specific events to be captured and used to trigger specific recovery commands to replicate, relocate, or restart containers as needed. This direction seeks to provide a single pane of glass, such as the System Controller Horizon GUI, for managing failover across geographically distributed Edge Clouds from a centralised location. A sensor fusion-like framework for combining health statistics from both Edge workloads and infrastructure can improve failover performance, support hardware and software predictive maintenance plans, and aid in the Edge workload placement process.
- Evaluate the failover performance of a larger StarlingX-based DCI implementation. Because StarlingX is geared toward massively distributed hardware management, the scalability and reliability of workload failover may be compromised. This direction seeks to determine whether the centralised approach is still viable or whether alternatives must be sought.
- Investigate the inter-collaboration of StarlingX-based Edge Clouds to enable workload failover between Edge Clouds if they are disconnected from the central Cloud controller. While Edge autonomy is well-established in StarlingX-based DCI, this direction aims to provide autonomy to a group of Edge Clouds acting as a self-managed DCI, i.e., one Edge Cloud can act as a central Cloud to another Edge Cloud. A failover manager of one Edge Cloud, for example, can collaborate with nearby Edge Clouds to relocate or offload workloads. In the event of a core network failure, a large segment of the DCI, for example, becomes unreachable.
- Compare StarlingX to other DCI virtualization platforms. For instance, Akraino Edge Stack, OpenStack Regions, and AZs. Consider the following factors in this direction: edge scalability, autonomy, and failover performance.

ACKNOWLEDGMENT

A significant portion of the StarlingX-based DCI implementation presented in this study is the result of extensive technical support from the StarlingX Community, most notably via Communications Mailing Lists, and Documentation. The authors would like to thank Ildiko Vancsa[24] from the Open Infrastructure Foundation, and also would like to thank the anonymous reviewers for their insightful comments and suggestions on how to improve the paper's quality.

REFERENCES

[1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
[2] (2021). *StarlingX, R4.0*. [Online]. Available: https://www.starlingx.io/
[3] B. Cohen, "Edge computing: Next steps in architecture, design and testing," OpenInfra Found., Edge Comput. Group, Openstack Found., Austin, TX, USA, Tech. Rep., Jun. 2020. Accessed: Sep. 8, 2022. [Online]. Available: https://www.openstack.org/use-cases/edge-computing/edge-computing-next-steps-in-architecture-design-and-testing/
[4] StarlingX. (2021). *Distributed Cloud Installation R4.0*. [Online]. Available: https://docs.starlingx.io/r/stx.5.0/deploy_install_guides/r4_release/distributed_cloud/index.html
[5] J. Hong, Y. G. Hong, X. D. Foy, M. Kovatsch, E. Schooler, and D. Kutscher, "IoT edge challenges and functions," Internet Eng. Task Force, Fremont, CA, USA, Tech. Rep. Internet-Draft draft-irtf-t2trg-iot-edge-03, Aug. 2021.
[6] P.-H. Tsai, H.-J. Hong, A.-C. Cheng, and C.-H. Hsu, "Distributed analytics in fog computing platforms using tensorflow and kubernetes," in *Proc. 19th Asia–Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2017, pp. 145–150.
[7] E. Huedo, R. S. Montero, R. Moreno-Vozmediano, C. Vázquez, V. Holer, and I. M. Llorente, "Opportunistic deployment of distributed edge clouds for latency-critical applications," *J. Grid Comput.*, vol. 19, no. 1, pp. 1–16, Mar. 2021.
[8] N. Wang, B. Varghese, M. Matthaiou, and D. S. Nikolopoulos, "ENORM: A framework for edge node resource management," *IEEE Trans. Services Comput.*, vol. 13, no. 6, pp. 1086–1099, Nov./Dec. 2020.
[9] L. Larsson, H. Gustafsson, C. Klein, and E. Elmroth, "Decentralized kubernetes federation control plane," in *Proc. IEEE/ACM 13th Int. Conf. Utility Cloud Comput. (UCC)*, Dec. 2020, pp. 354–359.
[10] M. A. Tamiru, G. Pierre, J. Tordsson, and E. Elmroth, "Instability in geo-distributed Kubernetes federation: Causes and mitigation," in *Proc. 28th Int. Symp. Modeling, Anal., Simulation Comput. Telecommun. Syst. (MASCOTS)*, Nov. 2020, pp. 1–8.
[11] S. Ishihara, S. Tanita, and T. Akiyama, "A dataflow application deployment strategy for hierarchical networks," in *Proc. IEEE 43rd Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, Jul. 2019, pp. 326–335.
[12] E. Kristiani, C. T. Yang, C. Y. Huang, Y. T. Wang, and P. C. Ko, "The implementation of a cloud-edge computing architecture using OpenStack and Kubernetes for air quality monitoring application," *Mobile Netw. Appl.*, vol. 26, pp. 1070–1092, Jul. 2020.
[13] Z. Benomar, F. Longo, G. Merlino, and A. Puliafito, "Cloud-based enabling mechanisms for container deployment and migration at the network edge," *ACM Trans. Internet Technol.*, vol. 20, no. 3, pp. 1–28, Oct. 2020.
[14] H. Koziolek, S. Gruner, and J. Ruckert, "A comparison of MQTT brokers for distributed IoT edge computing," in *Proc. Eur. Conf. Softw. Archit.* Springer, 2020, pp. 352–368.
[15] European Telecommunications Standards Institute, *Mobile Edge Computing (MEC); Mobile Edge Management; Part 1: System, Host and Platform Management*, document GS MEC 010-1 V1.1.1, Oct. 2017.

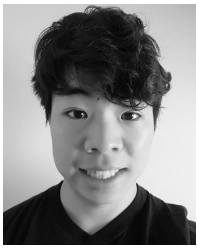
[24]https://github.com/ildikov

[16] *Network Functions Virtualisation (NFV); Architectural Framework*, document GS NFV 002 V1.2.1, Dec. 2014.
[17] A. Mohammed, G. Amirhossein, S.-M. Aidan, Y. Owen, Y. Thomas, and S. Marc. (Jun. 2021). *StarlingX R4.0 Virtual All-in-One Duplex Distributed Cloud Infrastructure Installation Guidelines*. [Online]. Available: https://github.com/MohammedAbuibaid/Stx_DCI_Failover/blob/master/Installation_Guidelines.md

MOHAMMED ABUIBAID received the B.Sc. degree (Hons.) in telecommunication engineering from An-Najah National University, Nablus, Palestine, in 2013, and the M.Sc. degree in electronic and communication engineering from Kocaeli University, Kocaeli, Turkey, in 2018. He is currently pursuing the Ph.D. degree with the Department of Systems and Computer Engineering, Carleton University, Ottawa, ON, Canada. From November 2016 to August 2019, he worked as a Technical Configurations Expert at Nokia, where he was in-charge of auditing Radio Access Network solutions. He worked as a Radio Frequency and Optimization Engineer at Ooredoo Palestine, from May 2013 to December 2014, where he was in-charge of tuning radio software features and dimensioning radio access network resources. His research interests include massive MIMO systems, multi-access edge computing, and time-sensitive networking.

AMIR HOSSEIN GHORAB received the B.Sc. degree in electrical engineering from Shahid Rajaee University, Tehran, Iran, in 2014, and the M.Sc. degree in computer engineering from the Iran University of Science and Technology, Tehran, in 2019. He is currently pursuing the Ph.D. degree with the Department of Systems and Computer Engineering, Carleton University, Ottawa, ON, Canada. From February 2015 to January 2019, he was a Software Programmer and a Research Assistant at the Iran University of Science and Technology. He developed SDN solutions running as a web application with Java and deploying cloud infrastructure with Openstack and Kubernetes. His research interests include cloud and edge computing, time-sensitive networking, SDN, virtualization technologies, and service function chaining.

AIDAN SEGUIN-MCPEAKE received the B.I.T. degree in network technology from Carleton University, in 2021. Since September 2021, he has been a Technical Support Engineer for Cloud Security at Verkada, San Mateo, CA, USA.



OWEN YUEN received the B.I.T. degree in network technology from Carleton University, in 2021. Since July 2022, he has been working as a Platform Specialist at Rogers Communication, Canada.

THOMAS YUNGBLUT received the B.I.T. degree in network technology from Carleton University, in 2021. Since May 2021, he has been a Systems Developer for the Government of Canada, Ottawa, ON, Canada.

MARC ST-HILAIRE (Senior Member, IEEE) received the Ph.D. degree in computer engineering from Polytechnique Montreal, Canada, in 2006. He is currently a Professor with the School of Information Technology with a cross-appointment with the Department of Systems and Computer Engineering, Carleton University, Ottawa, Canada. His research interests include wired and wireless communications, mobile computing and networking, and cloud/fog computing. With more than 140 technical publications, his work has been published/presented in several journals and international conferences. In addition to serving as a member of technical program committees of various conferences, he is equally involved in the organization of several national and international conferences and workshops.